

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1008	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	K.Bhavana	Reg. No.:	192524178
Date:	3-08-2026	Duration:	60 minutes

Code of Conduct

I K . B h a v a n a (1 9 2 5 2 4 1 7 8) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____



Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Case Study 33 – SDG 5: Women Entrepreneurship Support Portal

Many women-owned small businesses struggle to access government schemes, financial assistance, and business mentoring. Existing information is fragmented across multiple websites. A government agency plans to create an integrated digital platform supporting registration, funding opportunities, training programs, and networking.

As a software engineer, analyze the above problem and propose a suitable software engineering solution.

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable SoftwareArchitecture

- Select an appropriate architectural style for the proposed system (e.g., LayeredArchitecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the SoftwareArchitecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss QualityAttributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. JustifyArchitectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

1. Introduction

1.1 Purpose of the Document

This document serves as the formal Architecture Design Report for the Women Entrepreneurship Support Portal, prepared as part of the CO2 Assessment Tool 2 for the Software Engineering course (CSA1008). Its purpose is to translate an informally stated problem statement into a rigorous, defensible software architecture, following the standard software engineering lifecycle activities of requirements analysis, architectural style selection, structural design, component specification, and quality attribute evaluation. The document is intended to be read by academic evaluators as well as by a hypothetical engineering team that would use it as the blueprint for implementation.

1.2 Scope of the System

The scope of the Women Entrepreneurship Support Portal covers the full lifecycle of an entrepreneur's interaction with the support ecosystem: onboarding and identity verification, business profile creation, discovery and application for government schemes, submission and tracking of funding requests, matching with mentors and enrolment in training programs, participation in a peer community, and secure document management. The scope also covers the administrative and institutional side of the platform: scheme publication and management by government officials, funding review and disbursement workflows for financial institutions, and platform-wide configuration, monitoring, and auditing by system administrators. Explicitly out of scope for this iteration are payment settlement logic beyond gateway integration, in-house identity verification (which is delegated to an external KYC provider), and advanced machine-learning-based scheme recommendation, which is identified as a future enhancement rather than a baseline requirement.

1.3 Stakeholders

- Women Entrepreneurs — the primary end users who register businesses, discover schemes, apply for funding, and engage with mentors and training.
- Government Officials — responsible for publishing scheme details, defining eligibility rules, and tracking applications for transparency and compliance.
- Financial Institutions — banks and lending partners who review funding applications, verify eligibility, and process disbursements.
- Mentors and Trainers — domain experts who guide entrepreneurs through structured mentoring relationships and deliver training programs.
- System Administrators — responsible for platform-wide user management, configuration, audit oversight, and operational health.

Each stakeholder group interacts with the system through a role-appropriate interface, but all groups ultimately share a single, consistent source of truth for entrepreneur, scheme, and funding data — a requirement that fundamentally shapes the architectural style selected in Section 3.

2. Analysis of System Requirements

2.1 Problem Overview and System Objectives

The Women Entrepreneurship Support Portal is envisioned as a unified digital platform that connects women entrepreneurs with government bodies, financial institutions, and mentors in order to simplify access to business registration, government schemes, funding, training, and networking support. The system must serve multiple stakeholder groups simultaneously while maintaining a single, consistent source of truth for entrepreneur, scheme, and funding data.

The primary objectives of the system are to:

- Provide a single-window portal where women entrepreneurs can register their businesses, discover applicable government schemes, and apply for funding without visiting multiple offices or portals.
- Enable government officials to publish, manage, and track scheme applications and disbursements transparently.
- Allow financial institutions to review funding applications, verify eligibility, and process approvals through secure integrations.
- Connect entrepreneurs with mentors and trainers through structured matching, training enrolment, and community/networking features.
- Give the system administrator full oversight of users, audit logs, and platform configuration.

2.2 Functional Requirements

Based on the problem statement, the following functional requirements were identified and directly map to the modules shown in the architecture diagram in Section 4:

- **User Registration & Login:** Support role-based registration and authentication for entrepreneurs, government officials, financial institutions, mentors/trainers, and the system administrator.
- **Entrepreneur Profile Management:** Allow entrepreneurs to create and maintain business profiles, including business category, stage, and documentation.
- **Government Scheme Management:** Enable officials to publish scheme details, eligibility rules, and deadlines, and allow entrepreneurs to browse and apply.
- **Funding Application Management:** Allow entrepreneurs to submit funding requests and financial institutions to review, approve, or reject them.
- **Business Mentoring Module:** Match entrepreneurs with suitable mentors based on business domain and stage, and manage mentoring sessions.
- **Training Program Management:** Allow trainers to create training programs and entrepreneurs to enrol and track progress.
- **Networking & Community Module:** Facilitate peer discussion, events, and community interaction among entrepreneurs.
- **Document Management:** Securely upload, store, and retrieve supporting documents (KYC, business proofs, certificates).
- **Notification Service:** Send real-time alerts for scheme deadlines, application status changes, and mentoring/training updates via email and SMS.
- **Reports & Analytics:** Generate dashboards for administrators and officials on scheme usage, funding disbursement, and platform engagement.

2.3 Non-Functional Requirements

- **Scalability:** The system must support a growing number of concurrent users across geographies as more entrepreneurs, schemes, and financial partners are onboarded.
- **Security:** Sensitive data such as KYC documents, financial details, and government scheme data must be protected with encryption, role-based access control, and secure API communication.
- **Performance:** Scheme eligibility checks, funding workflows, and mentor matching must respond quickly even under concurrent load, particularly during scheme application windows.
- **Reliability & Availability:** The portal must remain available for time-critical activities such as scheme deadlines and funding submissions, with minimal downtime.
- **Maintainability:** Individual services (scheme management, funding, mentoring, training) should be independently updatable as government policies and business rules evolve.
- **Interoperability:** The system must integrate with external services such as government scheme APIs, banking/payment gateways, identity verification, email, and SMS gateways through secure REST APIs.
- **Auditability:** All critical actions (approvals, disbursements, document access) must be logged for compliance and audit purposes.

2.4 Requirements Traceability

The table below maps each functional requirement to the primary non-functional concerns it raises and the architectural layer that ultimately addresses it. This traceability view ensures that no requirement identified during analysis is left unaddressed in the eventual design.

Functional Requirement	Key NFR Concerns	Primary Layer(s)
User Registration & Login	Security, Performance	Application Services, Data
Entrepreneur Profile Mgmt	Maintainability, Security	Application Services, Data
Government Scheme Mgmt	Scalability, Auditability	Application Services, Business Logic
Funding Application Mgmt	Reliability, Security, Auditability	Business Logic, External Services
Business Mentoring Module	Performance, Maintainability	Business Logic, Data
Training Program Mgmt	Scalability	Application Services, Data
Networking & Community	Availability	Application Services, Data
Document Management	Security, Scalability	External Services (Cloud Storage)
Notification Service	Reliability, Performance	Business Logic, External Services
Reports & Analytics	Performance, Availability	Business Logic, Data

2.5 Key Quality Attributes Driving the Architecture

Given the presence of multiple independent stakeholder groups (entrepreneurs, government, banks, mentors), the need for secure external integrations, and the requirement for auditable financial workflows, the architecture must specifically prioritize scalability (to add new schemes/services), security (financial and personal data), maintainability (evolving government policy), and reliability (funding and scheme deadlines

cannot be missed). These four attributes act as the primary design drivers referenced throughout the remainder of this report; performance and availability are treated as secondary but essential attributes that the primary drivers must not compromise.

3. Selection of Suitable Software Architecture

3.1 Selected Architectural Style

A Layered (N-Tier) Architecture, enhanced with Service-Oriented and event-based integration for external systems, has been selected for the Women Entrepreneurship Support Portal. The system is organized into five clearly separated layers — Presentation/Client, Application Services, Business Logic, Data, and an integrated External Services layer accessed through secure REST APIs — as shown in the architecture diagram in Section 4. An internal Event Broker sits between the Application Services and Business Logic layers to decouple request handling from downstream processing for high-fan-out actions such as notifications and reporting, giving the design a hybrid character without abandoning the overall layered discipline.

3.2 Justification for Selection

- **Clear separation of concerns:** Client applications (Web, Mobile, Admin Dashboard) are decoupled from business logic engines (eligibility verification, funding approval workflow, scheme recommendation), which in turn are decoupled from data storage — allowing each layer to evolve independently.
- **Multiple stakeholder-facing interfaces:** Entrepreneurs, officials, and administrators each require different views of the same underlying data; a layered design lets one Application Services layer serve all client types consistently.
- **Structured, workflow-driven processes:** Government scheme approval, funding disbursement, and eligibility verification are rule-based, sequential workflows that map naturally onto dedicated business-logic engines rather than loosely coupled event streams.
- **Controlled external integration:** Because the system must connect with government, banking, identity, and communication providers, isolating these integrations behind a secure API boundary (rather than exposing the whole system) reduces risk and simplifies compliance.
- **Simpler operations for the expected scale:** A full microservices/event-driven approach would add operational overhead (service discovery, distributed transactions) that is not justified at the current scale of a scheme/funding portal with well-defined, mostly synchronous workflows; the layered style with a service-integration layer meets the requirements with lower complexity.

3.3 Trade-off Considered

A pure microservices architecture would offer finer-grained independent scaling of each module, but was not selected as the primary style because it introduces higher operational complexity (service orchestration, distributed data consistency) that outweighs the benefit at the current scope. The design remains hybrid-friendly: any of the Layer 4 engines (e.g., Funding Approval Workflow, Notification Processor) can later be extracted into an independent microservice without redesigning the whole system, since they are already logically isolated behind well-defined interfaces.

3.4 Comparison of Candidate Architectural Styles

Before finalizing the layered approach, four candidate styles were evaluated against the key quality attributes identified in Section 2.5. The comparison below summarizes the reasoning that led to the final selection.

Architectural Style	Strengths for This Problem	Weaknesses for This Problem
Layered (N-Tier) — Selected	Clear separation of concerns; matches workflow-driven processes; simpler ops; supports multiple client types	Coarser-grained scaling than microservices
Microservices	Fine-grained independent scaling and deployment	High operational overhead; distributed transactions; not justified at current scale
Pure Event-Driven	Excellent decoupling for asynchronous notifications	Poor fit for sequential, auditable approval workflows
Monolithic	Simple to build initially	Tight coupling; hard to evolve scheme rules independently; high regression risk

4. Software Architecture Diagram

The diagram below illustrates the complete five-layer architecture of the Women Entrepreneurship Support Portal, along with the external services it integrates with via secure REST APIs. Information flows top-down from the user-facing layer through the application and business layers into the data layer, while cross-cutting concerns such as notification dispatch and audit logging are triggered through the Event Broker.

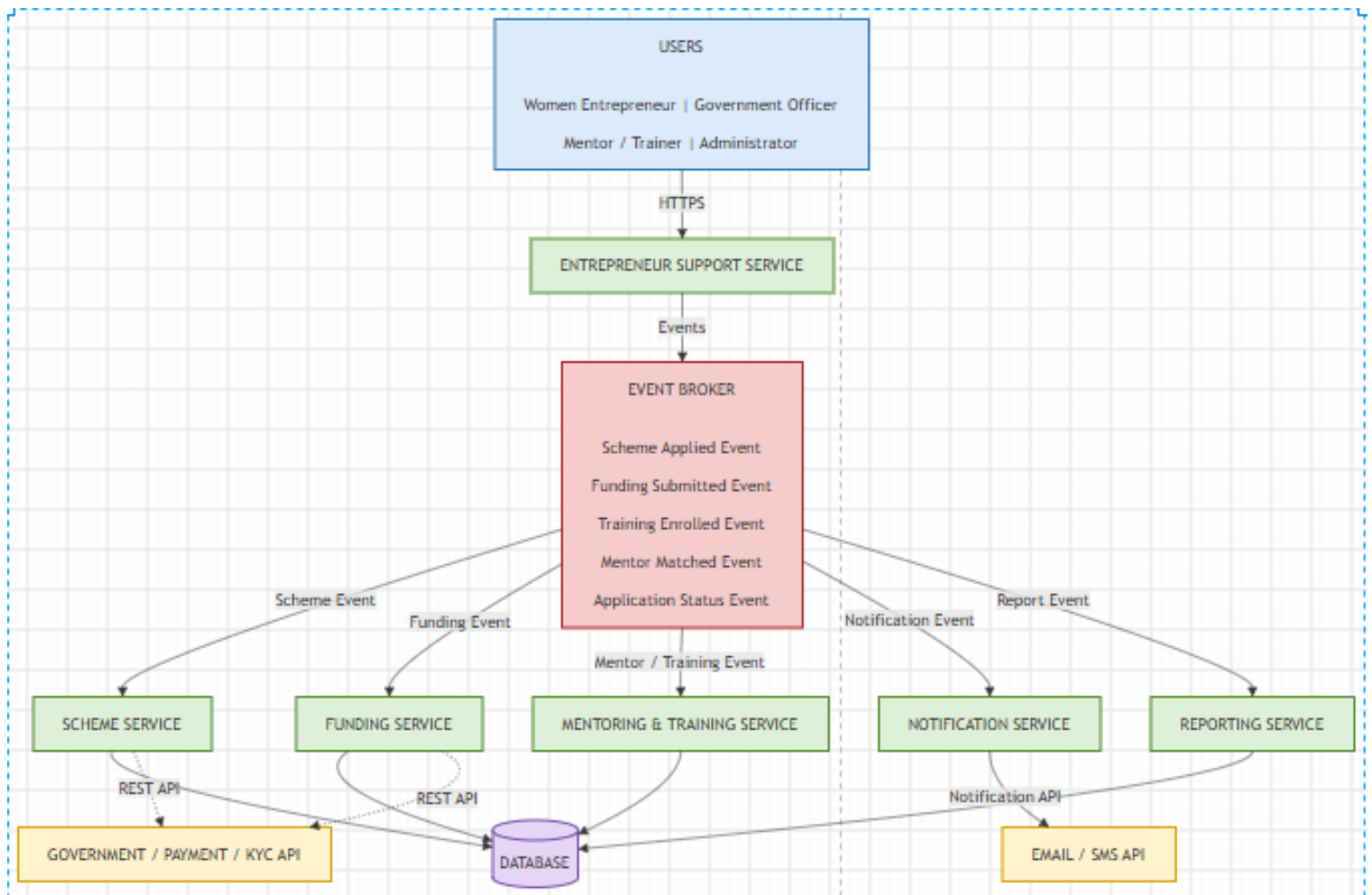


Figure 1: Layered Software Architecture of the Women Entrepreneurship Support Portal

The notation used follows conventional informal architecture-diagram practice: rounded rectangles represent components or services, arrows represent request/response or event flow, and grouped rows represent architectural layers. Colour is used purely to visually group each layer and does not carry additional semantic meaning.

5. Explanation of Components and Their Interactions

5.1 Layer 1 — Users

This layer represents the five actor types that interact with the system: Women Entrepreneurs (primary users seeking schemes, funding, mentoring, and training), Government Officials (who manage and approve schemes), Financial Institutions (who review and disburse funding), Mentors & Trainers (who guide entrepreneurs and run training programs), and the System Administrator (who manages platform-wide configuration, users, and audit logs). Each actor is routed to the appropriate client application based on role.

5.2 Layer 2 — Client Applications

The Web Portal and Mobile Application provide entrepreneur- and public-facing access, while the Admin Dashboard gives government officials, financial institutions, mentors, and administrators a management-focused interface. All client applications communicate with the Application Services layer over secure HTTPS/REST calls, keeping presentation logic separate from business logic.

5.3 Layer 3 — Application Services

This layer exposes the system's core services as API endpoints: User Registration & Login, Entrepreneur Profile Management, Government Scheme Management, Funding Application Management, Business Mentoring Module, Training Program Management, Networking & Community Module, Document Management, Notification Service, and Reports & Analytics. Each service receives requests from the client layer, validates them, and delegates business rules to the corresponding engine in Layer 4.

5.4 Layer 4 — Business Logic Engines

The business layer contains the rule-driven engines that implement the system's core decision-making:

- **Eligibility Verification Engine** — checks whether an entrepreneur qualifies for a scheme.
- **Funding Approval Workflow** — manages the multi-step approval process with financial institutions.
- **Scheme Recommendation Engine** — suggests relevant schemes based on the entrepreneur's profile.
- **Mentor Matching Engine** — pairs entrepreneurs with suitable mentors.
- **Training Enrolment Processor** — manages course enrolment and progress.
- **Analytics Engine** — computes usage and performance metrics.
- **Notification Processor** — generates and dispatches alerts.
- **Audit & Compliance Manager** — records all critical actions for governance.

5.5 Layer 5 — Data Layer

Each major domain has a dedicated data store — User Database, Business Profile Database, Funding Database, Government Scheme Database, Training Database, Mentor Database, Document Storage, and Audit Log Database — which keeps data ownership clear, supports independent scaling of each store, and simplifies data governance and backup strategy.

5.6 External Services and Integration

The portal integrates with six categories of external services through secure REST APIs: the Government Scheme API (to synchronize official scheme data), the Banking & Payment Gateway (for funding disbursement), Email and SMS Gateways (for notifications), an Identity Verification Service (for KYC/

eligibility checks), and Cloud Storage (for scalable document storage). All external communication is depicted as secure, authenticated REST calls, isolating the core system from direct exposure to third-party infrastructure.

5.7 Overall Workflow

A typical end-to-end flow begins when a Women Entrepreneur registers via the Web Portal or Mobile Application (Layer 2), which calls the User Registration & Login and Entrepreneur Profile Management services (Layer 3). When she applies for a scheme, the Government Scheme Management service invokes the Eligibility Verification Engine (Layer 4), which may call the Identity Verification Service and Government Scheme API (External Services) before reading/writing to the Government Scheme Database and Business Profile Database (Layer 5). A funding request follows a similar path through Funding Application Management, the Funding Approval Workflow engine, and the Banking & Payment Gateway, with every step recorded by the Audit & Compliance Manager. Throughout, the Notification Processor triggers Email/SMS alerts to keep the entrepreneur, officials, and mentors informed of status changes.

5.8 Sequence Walkthrough — Funding Application Example

To illustrate component interaction concretely, consider the funding application journey step by step:

- Step 1 — The entrepreneur submits a funding request from the Web Portal or Mobile Application.
- Step 2 — The request reaches the Funding Application Management service in the Application Services layer, which validates required fields and attached documents.
- Step 3 — The service publishes a Funding Submitted event to the Event Broker.
- Step 4 — The Funding Approval Workflow engine consumes the event, checks eligibility via the Eligibility Verification Engine, and calls the Identity Verification Service and Banking & Payment Gateway as needed.
- Step 5 — The Funding Database is updated with the application's current status, and the Audit & Compliance Manager records the transaction.
- Step 6 — The Notification Processor sends a status update to the entrepreneur and the reviewing financial institution via the Email/SMS Gateway.
- Step 7 — The Reports & Analytics service later aggregates this transaction into scheme usage and disbursement dashboards for administrators and officials.

6. Discussion of Quality Attributes

6.1 Scalability

Because each Application Service (Layer 3) and Business Engine (Layer 4) is logically isolated with its own dedicated database (Layer 5), high-demand modules such as Government Scheme Management or Funding Application Management can be scaled horizontally (e.g., adding more instances) during peak periods such as scheme deadlines, without needing to scale the entire platform. The Event Broker additionally absorbs bursts of activity — for example, a surge of scheme applications on a deadline day — by buffering events for downstream engines to process at a sustainable rate, rather than forcing every request through a single synchronous chain.

6.2 Security

All client-to-server and server-to-external-service communication happens over secure REST APIs, as explicitly marked in the diagram. Role-based access separates entrepreneur, official, financial institution, mentor, and administrator permissions. The Identity Verification Service supports KYC-based authentication, and the Audit & Compliance Manager combined with the Audit Log Database ensures traceability of every sensitive action such as funding approvals and document access. Transport-level encryption (TLS) and token-based authentication (JWT/OAuth2) further ensure that credentials and payloads are protected in transit, while encryption at rest protects KYC documents and financial records held in the Data layer.

6.3 Performance

Splitting business logic into dedicated engines (e.g., Eligibility Verification Engine, Scheme Recommendation Engine) allows performance-critical operations to be optimized and cached independently — for instance, scheme recommendations can be pre-computed, while eligibility checks remain synchronous and fast, keeping user-facing response times low even during high-traffic application windows. Read-heavy operations such as browsing schemes or viewing dashboards can additionally be served from cached or replicated read paths without affecting the write-heavy funding and scheme-application workflows.

6.4 Reliability

Dedicated engines for approval workflows and notifications, combined with an Audit & Compliance Manager, ensure that funding and scheme-related transactions are tracked end-to-end and can be recovered or reconciled if a step fails, rather than being lost in a monolithic process. The Event Broker provides an additional reliability guarantee: if a downstream engine (such as the Notification Processor) is temporarily unavailable, the triggering event can be retried or replayed rather than being silently lost.

6.5 Maintainability

The layered structure allows government policy changes (e.g., new scheme rules) to be implemented by updating the Government Scheme Management service and Eligibility Verification Engine alone, without impacting unrelated modules such as Mentoring or Training. This isolation significantly reduces regression risk when requirements evolve, and it allows different engineering sub-teams to own different Layer 4 engines with minimal coordination overhead.

6.6 Availability

Separating data stores per domain (User, Funding, Scheme, Training, Mentor, Document, Audit) means an issue in one store (e.g., Training Database maintenance) does not bring down critical services like Funding Application Management or Government Scheme Management, improving overall platform availability during

partial outages or maintenance windows. Stateless Application Services further allow multiple instances to be deployed behind a load balancer so that the failure of a single instance does not interrupt service.

6.7 Quality Attribute Summary Matrix

Quality Attribute	Architectural Mechanism	Primary Layer(s) Responsible
Scalability	Independent service/database scaling; event buffering	Application Services, Data, Event Broker
Security	TLS, RBAC, JWT/OAuth2, KYC integration, audit logging	All layers; External Services
Performance	Dedicated engines; caching; pre-computed recommendations	Business Logic
Reliability	Event replay/retry; Audit & Compliance Manager	Event Broker, Business Logic
Maintainability	Layer isolation; narrow service interfaces	Application Services, Business Logic
Availability	Domain-specific data stores; stateless services; load balancing	Data, Application Services

7. Justification of Architectural Decisions

7.1 Rationale

The layered architecture with an isolated external-services boundary was chosen because the problem domain is fundamentally workflow- and data-integrity-driven (scheme applications, funding approvals, compliance) rather than a high-frequency, loosely-coupled event stream. Clear separation between presentation, application services, business engines, and data stores directly supports the non-functional priorities identified in Section 2 — security, maintainability, and auditability — while still allowing scalability through independent scaling of individual services and databases.

7.2 Trade-offs Considered

- **Layered vs. Microservices:** Microservices would allow finer-grained independent deployment and scaling, but at the cost of higher operational complexity (service discovery, distributed transactions, network latency) that is not justified for the current scope. The chosen layered design keeps the logical separation microservices would provide (each Layer 4 engine has a clear, narrow responsibility) while avoiding premature distributed-systems overhead.
- **Monolithic vs. Layered:** A single monolithic application would be simpler to build initially but would tightly couple scheme management, funding, mentoring, and training logic, making it harder to update government policy rules without risking regressions elsewhere — this is why a layered separation was preferred over a monolith.
- **Synchronous vs. Event-Driven notifications:** The Notification Processor could be built as a fully event-driven, asynchronous component. This trade-off favors decoupling (entrepreneurs are notified without blocking core workflows) while keeping the rest of the system synchronous and easier to reason about.

7.3 Support for Future Enhancements and Long-Term Maintainability

The architecture supports future growth in several concrete ways. New government schemes or financial partners can be onboarded by extending the Government Scheme API and Banking & Payment Gateway integrations without altering core application logic. Individual Layer 4 engines (e.g., Scheme Recommendation Engine) can be independently upgraded — for example, replaced with a machine-learning-based recommender — without impacting other engines, because they are already isolated behind clear interfaces. Additional client channels (e.g., a chatbot or WhatsApp interface) can be added at Layer 2 and would reuse the same Application Services layer, avoiding duplication of business logic. Finally, because each data store is domain-specific, new modules (e.g., a Grants Marketplace) can be added as a new vertical slice — new client entry point, application service, business engine, and database — with minimal disruption to the existing system, ensuring the architecture remains maintainable as the platform scales over time.

8. Assumptions and Constraints

8.1 Assumptions

- Government scheme data is made available through an official, documented API rather than manual data entry.
- Financial institutions expose (or agree to expose) a secure banking/payment gateway API for funding disbursement rather than requiring manual reconciliation.
- All five actor types (entrepreneurs, officials, financial institutions, mentors/trainers, administrators) access the system over a reasonably stable internet connection via web or mobile.
- Identity verification (KYC) is handled by a dedicated external service rather than being built in-house.
- The initial user base is moderate (regional/state-level rollout) and can grow to a national scale over time, which the layered design must accommodate without a full redesign.

8.2 Constraints

- The system must comply with government data-protection and financial-data regulations, which constrains where and how KYC, banking, and scheme data can be stored and transmitted.
- All integrations with external parties (government, banks, identity providers) are limited to secure REST APIs, since direct database-to-database integration with third parties is not permitted.
- The project timeline and team size favor a layered architecture over a distributed microservices approach, which would require additional DevOps maturity (service orchestration, monitoring) not assumed to be available at this stage.
- Document storage must scale to a large and growing volume of KYC and business proof documents, which constrains the choice of storage to scalable cloud storage rather than local/on-premise storage alone.

9. Tools and Technologies Recommended for Implementation

While the assignment focuses on architectural design rather than implementation, the following technology categories are recommended as a realistic basis for building each layer of the proposed architecture:

Layer / Component	Representative Technology Options
Client Applications	React / Angular for the Web Portal; Flutter or React Native for the Mobile Application; a React-based Admin Dashboard
Application Services	REST APIs built with Node.js (Express) or Java Spring Boot, exposed over HTTPS
Business Logic Engines	Java/Spring or Python services encapsulating eligibility, approval, and matching rules; a rules engine (e.g., Drools) for eligibility/approval logic

Layer / Component	Representative Technology Options
Data Layer	PostgreSQL/MySQL for relational data (users, schemes, funding); MongoDB or cloud object storage for documents
External Integrations	Government Scheme API and Identity Verification via secure REST/OAuth2; payment gateway SDKs (e.g., Razorpay/UPI-compliant gateways); SMTP/email API and SMS gateway providers
Security & Auditing	JWT/OAuth2 for authentication, TLS for all API traffic, and centralized logging (e.g., ELK stack) for the Audit & Compliance Manager
Deployment	Containerized deployment (Docker) on a cloud platform, enabling any Layer 4 engine to be extracted into an independent microservice later if needed

10. Risk Analysis

A brief risk assessment complements the architectural design by identifying areas where the chosen approach carries residual risk, along with the corresponding mitigation built into the design.

Risk	Likelihood	Impact	Mitigation
External government/banking API downtime	Medium	High	Isolate integrations behind External Services boundary; queue and retry via Event Broker
Sudden spike in applications near scheme deadlines	High	Medium	Horizontal scaling of Application Services; event buffering
Data breach of KYC/financial documents	Low	High	Encryption at rest and in transit; RBAC; dedicated Document Storage with access logging
Regression when scheme rules change	Medium	Medium	Isolated Eligibility Verification Engine with narrow interface
Vendor lock-in with a single payment gateway	Low	Medium	Gateway integration abstracted behind the Funding Approval Workflow engine

11. Conclusion

The Women Entrepreneurship Support Portal addresses a genuinely multi-stakeholder problem — connecting entrepreneurs, government bodies, financial institutions, and mentors — that demands a clear separation between presentation, business rules, and data, together with tightly controlled external integrations. The selected layered architecture, enhanced with a dedicated external-services boundary and an internal event broker, satisfies these needs directly: it isolates sensitive financial and government-scheme workflows behind

well-defined business engines, keeps each data domain independently manageable, and exposes a consistent set of application services to all client types.

The design was deliberately chosen over a full microservices architecture to avoid unjustified operational complexity at the current scale, while still preserving the modularity needed for future growth — new schemes, financial partners, mentoring features, or entirely new modules can be added as vertical slices without disrupting existing functionality. Taken together, the architecture meets the core quality attributes identified during requirements analysis — scalability, security, performance, reliability, maintainability, and availability — while remaining practical to build, operate, and evolve over the platform's lifetime.

References

- SIMATS Engineering, CO2 Assessment Tool 2 — Architecture Design Assignment, Course CSA1008, Software Engineering.
- Bass, L., Clements, P., & Kazman, R. Software Architecture in Practice. Addison-Wesley.
- Fowler, M. Patterns of Enterprise Application Architecture. Addison-Wesley.
- Newman, S. Building Microservices. O'Reilly Media. Richardson, C. Microservices Patterns. Manning Publications.

Rubrics for Calculation

Ev alu at io n Cri ter ia	Excell ent (4 Mark s)	Goo d (3 Mar ks)	Satisfa cto r y (2 Marks)	Need s Impr ove me nt (1 Mar k)
System Requireme nt Analysis	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requiremen ts with partial analysis.	Requirem e nt analysis is incomplet e, inaccurate , or lacks clarity.
Selection of Software Architectu re	Selects the most appropriate architectural style	Selects a suitable architecture with reasonable justification.	Architectur is partially suitable with limited	Inappropria e architecture selected or justification

	with strong technical justification to system requirements.		justification.	is missing.
	Produces a clear,		Diagram	
Architecture Diagram and Component Design	complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
	Clearly explains component			Component

Component Interaction and Quality Attribute Analysis	responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	interactions and quality attributes are poorly explained.
Architectural Justification and Technical Presentation	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional	Provides adequate justification with minor gaps; report is well	Limited justification with minimal discussion of trade-offs or future scope; report	Justification is weak or absent; report lacks organization and techni

	report.		needs improvement.	cal clarity
--	---------	--	-----------------------	----------------

Criteria	Mark
System RequirementAnalysis	/5
Selection of SoftwareArchitecture	/5
Architecture Diagram and Component Design	/5
Component Interaction and QualityAttributeAnalysis	/5
Architectural Justification and Technical Presentation	/5
TOTAL	/5